VEHICLE RENTAL

SYSTEM

In this project we will create a vehicle rental system using Java ,the System will allow users to rent different types of vehicles and manage their rentals

such as vehicles is a base class the Car and motorcycles are subclasses

BASE CLASS

```
public abstract class Vehicle {
private int vehicleId;
private String make;
private String model;
private int year;
private boolean rentalStatus;
private double rentalRate;
// Constructor
public Vehicle(int vehicleId, String make, String model, int year,
boolean rentalStatus, double rentalRate) {
this.vehicleId = vehicleId;
this.make = make;
this.model = model;
this.year = year;
this.rentalStatus = rentalStatus;
this.rentalRate = rentalRate;
}
// Getters and setters
public int getVehicleId() {
return vehicleId;
}
public void setVehicleId(int vehicleId) {
this.vehicleId = vehicleId;
}
public String getMake() {
return make;
}
public void setMake(String make) {
this.make = make;
}
```

```
public String getModel() {
return model;
}
public void setModel(String model) {
this.model = model;
}
public int getYear() {
return year;
}
public void setYear(int year) {
this.year = year;
}
public boolean isRentalStatus() {
return rentalStatus;
}
public void setRentalStatus(boolean rentalStatus) {
this.rentalStatus = rentalStatus;
}
public double getRentalRate() {
return rentalRate;
}
public void setRentalRate(double rentalRate) {
this.rentalRate = rentalRate;
}
// Abstract method to calculate rental cost
public abstract double calculateRentalCost();
@Override
public String toString() {
return "Vehicle [vehicleId=" + vehicleId + ", make=" + make + ",
model=" + model + ", year=" + year
+ ", rentalStatus=" + rentalStatus + ", rentalRate=" + rentalRate + "]";
}
```

EXTENDS CLASSES

```
public class Car  extends Vehicle {
private int numSeats;
private int numDoors;

// Constructor
public Car(int vehicleId, String make, String model, int year,
boolean rentalStatus, double rentalRate, int numSeats, int
numDoors) {
super(vehicleId, make, model, year, rentalStatus, rentalRate);
this.numSeats = numSeats;
this.numDoors = numDoors;
}
// Getters and setters
public int getNumSeats() {
return numSeats;
}
public void setNumSeats(int numSeats) {
this.numSeats = numSeats;
}
public int getNumDoors() {
return numDoors;
}
public void setNumDoors(int numDoors) {
this.numDoors = numDoors;
}
@Override
public double calculateRentalCost() {
return getRentalRate() * 1.5; // Example calculation
}
@Override
public String toString() {
return "Car [numSeats=" + numSeats + ", numDoors=" +
numDoors + "]";
}

}
```

```
public class Motorcycle  extends Vehicle {
private int engineCapacity;

// Constructor
public Motorcycle(int vehicleId, String make, String model, int
year, boolean rentalStatus, double rentalRate, int engineCapacity)
{
super(vehicleId, make, model, year, rentalStatus, rentalRate);
this.engineCapacity = engineCapacity;
}

// Getters and setters
public int getEngineCapacity() {
return engineCapacity;
}

public void setEngineCapacity(int engineCapacity) {
this.engineCapacity = engineCapacity;
}

@Override
public double calculateRentalCost() {
return getRentalRate() * 1.2; // Example calculation

}

}
```

ABSTRACT CLASS

```
import java.util.ArrayList;
import java.util.List;

public abstract class Customer {
// Add properties: customerId, name, rentedVehicles (list of vehicles)
private int customerId;
private String name;
private List<Vehicle> rentedVehicles;
// Add constructor to initialize the properties
public Customer(int customerId, String name) {
this.customerId = customerId;
this.name = name;
this.rentedVehicles = new ArrayList<>();

}
// Add getters and setters for the properties
public int getCustomerId() {
return customerId;
}
public void setCustomerId(int customerId) {
this.customerId = customerId;
}
public String getName() {
return name;
}
public void setName(String name) {
this.name = name;
}
public List<Vehicle> getRentedVehicles() {
return rentedVehicles;

}
}
```

```
// Add methods: rentVehicle(Vehicle vehicle), returnVehicle(Vehicle vehicle), displayRentedVehicles()
public void rentVehicle(Vehicle vehicle) {
if (!vehicle.isRentalStatus()) {
vehicle.setRentalStatus(true);
rentedVehicles.add(vehicle);
System.out.println(vehicle.getMake() + " " + vehicle.getModel() + " rented successfully to " +
this.name);
} else {
System.out.println("Vehicle is already rented");
}
}
public void returnVehicle(Vehicle vehicle) {
if (rentedVehicles.contains(vehicle)) {
vehicle.setRentalStatus(false);
rentedVehicles.remove(vehicle);
System.out.println(vehicle.getMake() + " " + vehicle.getModel() + " returned by " + this.name);
} else {
System.out.println("Vehicle is not rented by this customer");
}
}
public void displayRentedVehicles() {
if (rentedVehicles.isEmpty()) {
System.out.println(this.name + " has no rented vehicles.");
} else {
System.out.println("Rented vehicles by " + this.name + ": ");
for (Vehicle vehicle : rentedVehicles) {
}
System.out.println("Vehicle");
}
```

```
// Subclasses of Customer: IndividualCustomer and
CorporateCustomer
public class IndividualCustomer  extends Customer {
private static final Vehicle[] rentedvehicles = null;
// Add properties: age, licenseNumber
private int age;
private String licenseNumber;
// Add constructor to initialize the properties
public IndividualCustomer(int customerId, String name, int age,
String licenseNumber) {
super(customerId, name);
this.age = age;
this.licenseNumber = licenseNumber;
}
// Override displayRentedVehicles() method to include customer
details
@Override
public void displayRentedVehicles() {
if ("rentedVehicles".isEmpty()) {
System.out.println("this.name" + " (" + this.age + " years old,
license number: " + this.licenseNumber +
") has no rented vehicles.");
} else {
System.out.println("Rented vehicles by " + "this.name" + " (" +
this.age + " years old, license number: "
+ this.licenseNumber + "): ");
for (Vehicle vehicle : rentedvehicles) {
System.out.println(vehicle);
}
}
}}
```

```
public class CorporateCustomer  extends Customer {
// Add properties: companyName, contactPerson
private String companyName;
private String contactPerson;
// Add constructor to initialize the properties
public CorporateCustomer(int customerId, String name, String
companyName, String contactPerson) {
super(customerId, name);
this.companyName = companyName;
this.contactPerson = contactPerson;
}
// Override displayRentedVehicles() method to include customer
details
@Override
public void displayRentedVehicles() {
if (getRentedVehicles().isEmpty()) {
System.out.println(this.companyName + " (contact person: " +
this.contactPerson + ") has no rented vehicles.");
} else {
System.out.println("Rented vehicles by " + this.companyName + "
(contact person: " +
this.contactPerson + "): ");
for (Vehicle vehicle : getRentedVehicles()) {
System.out.println(vehicle);
}
}
}
}
```

```
// Subclasses of Customer: IndividualCustomer and
CorporateCustomer
public class IndividualCustomer  extends Customer {
private static final Vehicle[] rentedvehicles = null;
// Add properties: age, licenseNumber
private int age;
private String licenseNumber;
// Add constructor to initialize the properties
public IndividualCustomer(int customerId, String name, int age,
String licenseNumber) {
super(customerId, name);
this.age = age;
this.licenseNumber = licenseNumber;
}
// Override displayRentedVehicles() method to include customer
details
@Override
public void displayRentedVehicles() {
if ("rentedVehicles".isEmpty()) {
System.out.println("this.name" + " (" + this.age + " years old,
license number: " + this.licenseNumber +
") has no rented vehicles.");
} else {
System.out.println("Rented vehicles by " + "this.name" + " (" +
this.age + " years old, license number: "
+ this.licenseNumber + "): ");
for (Vehicle vehicle : rentedvehicles) {
System.out.println(vehicle);
}
}
}}
```

```
public class CorporateCustomer  extends Customer {
// Add properties: companyName, contactPerson
private String companyName;
private String contactPerson;
// Add constructor to initialize the properties
public CorporateCustomer(int customerId, String name, String
companyName, String contactPerson) {
super(customerId, name);
this.companyName = companyName;
this.contactPerson = contactPerson;
}
// Override displayRentedVehicles() method to include customer
details
@Override
public void displayRentedVehicles() {
if (getRentedVehicles().isEmpty()) {
System.out.println(this.companyName + " (contact person: " +
this.contactPerson + ") has no rented vehicles.");
} else {
System.out.println("Rented vehicles by " + this.companyName + "
(contact person: " +
this.contactPerson + "): ");
for (Vehicle vehicle : getRentedVehicles()) {
System.out.println(vehicle);
}
}
}
}
```

MAIN class

```
public class Main {
public static void main(String[] args) {
try {
// Create a car object
Vehicle car = new Car(1, "Toyota", "Corolla", 2020, true, 50.0,
5, 4);
// Create a motorcycle object
Vehicle motorcycle = new Motorcycle(2, "Honda", "CBR500R",
2021, false, 30.0, 500);
// Create customer objects
IndividualCustomer customer1 = new IndividualCustomer(101,
"John Smith", 35,"DL12345");
CorporateCustomer customer2 = new
CorporateCustomer(202, "ABC Company", "Jane Doe", "Manager");
// Rent vehicles to customers
customer1.rentVehicle(car);
customer2.rentVehicle(motorcycle);
// Display rented vehicles for each customer
customer1.displayRentedVehicles();
customer2.displayRentedVehicles();
// Calculate and print rental cost for car and motorcycle
System.out.println(motorcycle.toString());

System.out.println("Rental Cost for Car: $" +
car.calculateRentalCost());
System.out.println("Rental Cost for Motorcycle: $" +
motorcycle.calculateRentalCost());

}
catch (Exception e) {
System.out.println("Error: " + e.getMessage() );

}}
```

the Output

```
<terminated> Main (2) [Java Application] C:\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_6
Vehicle is already rented
Honda CBR500R rented successfully to ABC Company
Rented vehicles by this.name (35 years old, license number: DL12345):
Error: Cannot read the array length because "IndividualCustomer.rentedvehicles" is null
```

THANK YOU